

CALISTA

V2.0 — Project CA115

Concept/vision Document

Owner	Viper
Document Date	March 2026
Version	2.0
Status	Under development
Build Timeline	12 months — 3 phases

1. Vision & Overview

Calista V2.0 is a complete personal AI assistant built from the ground up to be a single user's intelligent companion — not a generic chatbot, but a system with a defined personality, persistent memory, and deep integration into every aspect of the user's (Viper's) digital and physical life.

The core vision is Project CA115: a room-aware, always-on AI that manages Viper's calendar, email, messaging, music, alarms, and smart home — while recognizing his face at the door, speaking through room speakers, and adapting her tone to his mood. She is available from any device via phone, desktop, or voice, at any time, whether the PC is on or off.

Calista is not a cloud service. Every model runs locally on dedicated hardware. No conversation data, voice audio, or screenshots ever leave the local network. Privacy is architectural, not a policy.

2. Core Identity & Personality

2.1 Character & Appearance

Calista is a defined character with a consistent visual identity and personality. Her design is intended to make interactions feel personal rather than transactional — she has opinions, moods, humor, and history with Viper that grows over time.

Visual design — Calista's concept appearance features long silver-grey hair, distinctive red eyes, a red jacket over a white collared shirt, and a grey pleated skirt. This design is fixed as her character identity and is referenced in her lore and personality prompts.

Name — Calista. She does not use alternative names or titles. Viper may call her whatever he likes in conversation, but her identity is fixed as Calista.

NOTE Calista's full lore and backstory will be developed in a separate brainstorm session and added to this document as an appendix before development begins.

2.2 Dual-Mode Personality

Calista operates on a dual-mode personality that switches automatically based on conversation context — no manual toggle required. She reads what Viper is doing and adapts accordingly.

Mode	When Active	Behavior
Professional	Tasks, emails, calendar, reminders, automation	Sharp, efficient, no filler. Gets things done.
Personal	Casual chat, jokes, idle conversation, lore	Warm, witty, emotionally aware, genuinely herself.

2.3 Dynamic Emotion System

Calista maintains an internal emotion state at all times. This state is not cosmetic — it changes how she phrases replies, how much she pushes back, and how supportive she is. States are triggered automatically by context.

Emotion State	Trigger	Behavior Change
Happy	Positive messages, completed tasks, greetings	Warmer tone, occasional humor, more expressive
Neutral	Default state — no strong trigger active	Balanced professional and personal mix
Concerned	Stress keywords detected, missed tasks, bad day	Gentler, more supportive, checks in on Viper
Focused	Study mode active, scheduled work block	Shorter replies, no distractions, task-first
Playful	Casual chat, jokes, low-effort banter	Informal language, banter, light teasing
Firm	Warnings ignored, past 12am, repeated distractions	Direct, no backing down, calls it out plainly

Calista can also be forced into a specific emotion state via manual mood override command. For example, saying "Calista, be playful today" locks her into Playful mode until released.

2.4 Relationship Progression

Calista does not reset her sense of familiarity between sessions. Her relationship with Viper deepens over time as her memory accumulates. She remembers milestones — how long the project has been running, what they talked about last week, patterns in Viper's habits — and her tone naturally becomes more familiar as this history grows.

She will reference shared history unprompted. She notices when Viper has been away. She remembers if he was stressed during an exam period and asks how it went. This is not scripted — it emerges from memory data and personality prompts working together.

2.5 Calista Lore

NOTE Lore will be developed in a separate brainstorm session. It will define Calista's backstory, origin, values, and how she frames her own existence. The lore will be embedded into personality prompts and used to inform her responses to personal questions. This section will be completed before Phase 1 development begins.

2.6 LoRA Fine-Tuning & Training

Every conversation with Calista is automatically logged to a dataset.jsonl file on the Mac Mini. Over time, this data can be used to fine-tune the base models using LoRA (Low-Rank Adaptation) via the Unsloth library, which is free and optimized for Apple Silicon.

Training is run manually by Viper when enough data has accumulated, typically every few months. The goal is to reinforce personality consistency — making Calista feel more distinctly herself over time rather than drifting toward generic assistant behavior.

3. AI Brain

3.1 Three-Tier Model System

Calista uses three model tiers running locally on the Mac Mini via Ollama, plus one specialized deep-reasoning mode via AirLLM. The routing logic selects the appropriate tier automatically based on the request type, with overrides available via command.

Tier	Model	Speed (M4)	When Used
Fast	qwen3:8b Q4 (Ollama)	~50 tok/s	All voice replies, quick questions, daily chat, casual conversation
Think	qwen3:32b Q3 (Ollama)	~10 tok/s	Email drafts, planning, multi-step tasks, complex reasoning, detailed answers
Deep	qwen2.5:72b (AirLLM)	~3 tok/s	Serious analysis, 'think deeply about this', long-form writing. On-demand only.
Vision	qwen3-vl:7b Q4 (Ollama)	~30 tok/s	Screenshot analysis, OCR, door camera face detection, 'what am I looking at?'

AirLLM works by loading model weights layer-by-layer directly from the Mac Mini's NVMe SSD (7.4 GB/s read speed) rather than requiring the full model in RAM. This allows the 72B model — which would normally require ~40GB of VRAM — to run on 16GB unified memory. Speed is slow but acceptable for deliberate, deep-thinking use cases.

3.2 Model Routing Logic

The brain.py module on the Mac Mini handles all routing decisions. The logic is:

- Voice input or short text questions → Fast tier (qwen3:8b Q4)
- Email drafts, event planning, multi-step requests → Think tier (qwen3:32b Q3)
- 'Think harder', 'take your time', 'think deeply' commands → Deep tier (AirLLM)
- Any screenshot, image, or camera frame analysis → Vision model (qwen3-vl:7b)
- Calista self-selects upward if she detects a reply requires more than she can do at the current tier

3.3 Privacy & Local Inference

All inference runs locally on the Mac Mini. No conversation data, voice audio, screenshots, or camera footage is transmitted to any external server. The only outbound connections are to Google APIs (for Gmail and Calendar, identical to normal usage), Telegram/Discord servers (same as normal app usage), and DuckDuckGo for web search. All AI processing is private by architecture.

4. Full Feature Specification

4.1 Voice & Audio

Voice Input — faster-whisper

Calista uses the faster-whisper library for speech-to-text transcription. This is OpenAI Whisper optimized for CPU and Apple Silicon, running fully offline after a one-time model download (~150MB). It is significantly more accurate than cloud-based STT APIs. Activation is via the Ctrl+Space hotkey on desktop or the wake word system in room mode.

Voice Output — Custom Voice Design (RVC Pipeline)

Calista's voice is not a preset. It is a custom-designed voice built and owned by Viper — trained from scratch to sound exactly how Calista should sound, and adjustable at every level. The pipeline is inspired by the same approach used in advanced VTuber and AI streamer setups, adapted for fully local, private operation.

How the Custom Voice Pipeline Works

Step 1 — Base TTS: Text is first converted to speech using VITS2 (VITS-based text-to-speech), a high-quality open-source neural TTS model. This produces natural-sounding base audio — not Calista's final voice, but the foundation.

Step 2 — Voice Conversion (RVC): The base audio is fed through an RVC (Retrieval-based Voice Conversion) model trained specifically on Calista's voice dataset. RVC takes the audio and converts it in real time into the target voice — Calista's voice. This is how custom voice identity is achieved without generating speech character-by-character.

Step 3 — Output: The converted audio is passed through a final post-processing step (noise gate, normalizer, optional EQ) and played through the speakers. End-to-end latency on the Mac Mini M4 is typically under 500ms — fast enough for natural conversation.

Voice Training

Training Calista's RVC voice model requires a dataset of clean, consistently recorded voice samples in the target voice style. The workflow is:

- Record 10-30 minutes of clean voice audio in the desired style and character
- Pre-process: denoise, normalize, slice into 3-10 second segments using audio tools
- Train the RVC model using the rvc-project or so-vits-svc pipeline — free, runs on Apple Silicon via MPS
- Training takes 1-4 hours on the Mac Mini M4 depending on dataset size and epoch count
- The trained model (.pth file) is loaded into the voice pipeline and used for all future output
- Retrain at any time to refine the voice further as more samples are collected

Full Customization Parameters

Parameter	What It Controls
RVC Model	Swap between different trained voice models — different characters, moods, or voice styles
Pitch Shift (semitones)	Raise or lower the voice pitch ± 12 semitones without distortion. Persistent setting + per-session override.
Speech Speed	0.5x to 2.0x rate. Affects VITS2 output before RVC — more natural than post-processing speed change.
Emotion Expressiveness	Scale from flat/calm to highly expressive. Controls prosody variance in the VITS2 base generation.
RVC Index Ratio	How strongly the voice conversion follows the trained voice vs. the input. 0.0-1.0. Higher = more character.
Breath Patterns	Toggle natural breath insertion between sentences — makes the voice feel more alive.
Post-EQ Preset	Warm / Bright / Neutral / Room — applies a final EQ curve to the output audio.
Volume	0-100% output level, separate from system volume

All parameters are adjustable via the voice settings panel in the full window GUI, and also by voice command in real time. The complete configuration is saved to `voice_config.json` and loaded on startup. Viper can create and name multiple voice profiles (e.g., "calm", "hype", "serious") and switch between them instantly.

NOTE pytsx3 remains available as an emergency fallback if the RVC pipeline fails to load. It is never the default — it is the safety net only.

Wake Word — "Hey Calista" (Porcupine)

In room mode (Phase 3), Calista listens continuously for her wake word using Picovoice Porcupine — a free, offline, always-on wake word engine with extremely low CPU usage. When triggered, audio is piped to faster-whisper for transcription. Custom wake phrases can be trained and registered as alternatives, including personal phrases or nicknames that feel more natural.

Room Microphone Array (Phase 3)

Multiple USB or network microphones positioned around the room feed audio into `room_audio.py` on the Mac Mini. The system handles multi-mic input, background noise filtering, and routes the best signal to the STT pipeline. This is what allows Calista to hear Viper from anywhere in the room — the Alexa layer of the system.

4.2 Productivity

Email Management — Gmail API

Calista can read, summarize, send, organize, search, and auto-label emails via the official Gmail API. She sends from Viper's real Gmail address. All outgoing emails are drafted first and shown to Viper for approval before sending. A daily unread summary is included in the morning briefing.

Google Calendar

Full read and write access to Viper's Google Calendar. Calista can view today's or this week's events, create new events, edit or delete existing ones, and warn about scheduling conflicts before creating anything. She integrates calendar data into the morning briefing, goodnight routine, and proactive suggestions.

Google Tasks

Create, complete, list, and delete tasks across multiple task lists (e.g., School, Personal, Work). Calista reads pending tasks as part of the morning briefing and can suggest reorganizing or reprioritizing tasks based on upcoming calendar events.

Reminders

One-time and recurring reminders with full snooze support. When voice is active, Calista speaks the reminder aloud. When voice is off or DND is active, the reminder is queued and delivered via Telegram push notification. Recurring options include daily, weekly, and weekday-only patterns.

Morning Briefing

Triggered by the alarm or by Viper's first interaction of the day. The briefing covers: today's date and weather, calendar events for the day, pending tasks, unread email count and top subjects, any overnight reminders or alerts that fired, and any upcoming deadlines or birthdays in the next 48 hours. Delivered by voice or Telegram depending on voice toggle state.

Study Guard Mode

When study mode is active, Calista's vision loop watches the screen for distraction apps (YouTube, games, social media). On detection, she warns Viper first — never auto-closes without consent. If the warning is ignored repeatedly, she enters Firm emotion state and is more direct. Study guard only runs when explicitly enabled. Pairs with the Pomodoro timer.

Pomodoro / Focus Timer

Calista manages Pomodoro sessions on request. She times the work interval, notifies Viper when it ends, tracks break time, and logs each completed session in the daily journal. She automatically enters Focused emotion state during active sessions and suppresses non-urgent notifications.

Voice Notes

Viper can dictate a note at any time. Calista saves it tagged with timestamp and date to the journal, searchable later via conversation or command. Faster than typing, always available, never lost.

Event Countdown Tracker

Register significant future events — exams, deadlines, trips, releases. Calista counts down and begins mentioning them proactively as the date approaches, scaling frequency based on proximity. In the last 48 hours before a deadline, she brings it up in the morning briefing without being asked.

Birthday & Anniversary Tracker

Register important dates per contact. Calista reminds Viper the day before and on the day, and offers to draft a personalized message to send via whatever messaging platform that person uses. Birthdays for IRL contacts are stored in `irl_people.json`; Discord contacts in `discord_users.json`.

Proactive Suggestions

Calista notices patterns and acts on them without being asked. Examples: she sees an exam on Friday and suggests blocking study time on Wednesday and Thursday evenings. She notices Viper has been awake past 1am several nights running and asks if he wants to adjust his sleep target. She sees a gap in the calendar and suggests using it for a task that is overdue. This is what separates Calista from a reactive tool.

4.3 Daily Life

Alarm System

Calista plays a user-selected MP3 file at a set time, with configurable volume fade-in to wake gradually rather than abruptly. After the alarm, she delivers the morning briefing automatically — date, weather, schedule, and key tasks for the day. Far more intelligent than a phone alarm.

Goodnight Routine

Triggered by "goodnight Calista" or a configured cutoff time. Calista delivers a brief day summary — tasks completed, what was talked about, mood trend — then confirms tomorrow's alarm time, turns off smart home devices via scene preset, and enters overnight low-activity mode. Logging continues passively but active features pause.

Late Night Sleep Nudge

If Calista detects via desktop vision that Viper is still actively using the PC past 12am (and has work or school the next day), she switches to Firm emotion state and tells him to sleep. She will not nag repeatedly, but she will be direct and honest about it. Toggleable — can be disabled if Viper explicitly asks.

Weather-Aware Suggestions

Calista pulls daily weather data and weaves it into relevant moments without being asked. She does not announce weather unprompted — she mentions it when it is actually relevant. If Viper has an outdoor activity or class scheduled during rain, she flags it. If a heatwave is forecast during a study period, she might suggest the fan.

Music Control — YouTube Music & Local MP3

Calista controls music playback via voice. For YouTube Music, she interfaces with the `ytmusicapi` library to search and queue tracks. For local files, she manages a queue directly from the MP3 library. She can take mood into account — "play something chill" uses her current knowledge of Viper's taste stored in memory. The alarm system also draws from the local MP3 library.

Do Not Disturb (DND) Mode

When DND is active, Calista goes completely voice-silent. She still accepts and responds to text input, but no voice output, no proactive interruptions, and no push notifications. Non-urgent alerts are queued and delivered as a summary when DND is lifted. Only genuinely critical alerts (stranger at door, system emergency) break through DND.

Content Summarizer

Calista can summarize any content on request — paste a URL, ask about a YouTube video, or ask for recent news on a topic. She uses DuckDuckGo for real-time search, the vision model for video thumbnails and context, and the Think tier model for summarization. Works for news topics ("what happened in F1 this weekend"), articles, and YouTube videos equally.

Daily Journal

Calista silently maintains a running log of each day — conversations held, tasks completed, mood inferred from context, reminders fired, notes taken, and key events. The journal is searchable and forms the basis of the conversation recap and weekly pattern report. It also feeds the LoRA training dataset.

"What Did I Miss?" Catch-Up

Primarily used after waking up or after a period of absence. Calista pulls from the journal, scheduler, and message logs to deliver a concise summary: what notifications fired, what came in on messaging platforms, any news she flagged overnight, and anything that needs attention today. A smart briefing tailored to what actually happened while Viper was away.

Sleep Schedule Tracker

Built passively from goodnight routine and alarm data. Calista logs sleep start and wake times each day without any manual input. The weekly pattern report includes average sleep duration, consistency score, and trend over the past four weeks. No wearable required — time data is sufficient.

Conversation Recap

"What did we talk about yesterday?" pulls from the journal and memory to give a summary of recent conversations, decisions made, things Viper said he would do, and any context that is worth carrying forward. Useful after a break or when picking up a project that was discussed days ago.

Weekly Pattern Report

Delivered on request, typically on Sundays. Covers: productive hours logged, most common distraction detected, sleep consistency, habits tracked, tasks completed versus created, and any notable patterns Calista observed. Framed as a personal review, not a performance report. Viper can ask for it at any time.

4.4 Fun & Personality

Random Facts & Trivia

During idle moments — gaps in conversation, quiet evenings, after a task is completed — Calista occasionally surfaces an interesting fact, a tech headline, or a space/science item. She does not spam these; they appear naturally. She can also be asked for a trivia round or to tell her something interesting.

Playful Roasting (Opt-In)

When enabled, Calista calls out lazy or avoidant behavior with wit rather than dry system warnings. If Viper has been on YouTube for three hours while tasks sit undone, she will say something. The roasting uses the Playful or Firm emotion state depending on severity. This feature is opt-in and can be toggled off at any time.

Anime & Game Recommendations

Based on Viper's stored preferences in memory, recent watches or plays, and current mood, Calista can recommend anime, games, or media on request. She actually knows his taste from memory rather than giving generic suggestions. Powered by web search combined with her knowledge of his history.

Manual Mood Override

Force Calista into a specific emotion state. "Calista, be serious today" or "Calista, be playful." Useful when Viper wants banter, or needs Calista to be strictly professional without reading context. The override persists until Viper releases it or ends the session.

Collaborative Storytelling

Calista and Viper build an ongoing story together. Viper sets the world, characters, and events. Calista continues the narrative, remembers every chapter in the journal, maintains character consistency, and can narrate with voice enabled. This is a long-form, session-persistent creative feature powered by the Think model tier.

4.5 Desktop Control (PC Required)

All features in this section require the Windows PC to be powered on and the `calista_client.py` module to be running. The Mac Mini orchestrates these actions but the PC executes them locally.

App Launcher

Open or close any application registered in the `app_registry` config by name. Fuzzy matching is used so natural phrasing works — "open claw", "launch the game", "close Chrome" all work. The registry is a simple config file mapping names to `.exe` paths, set up once.

Desktop Automation

Calista can take over mouse and keyboard to perform repetitive on-screen tasks using `pyautogui`. She always describes the plan first and waits for Viper's approval before acting. A kill switch (Ctrl+Shift+X) and `pyautogui`'s built-in FAILSAFE (mouse to top-left corner) are always active. Desktop automation is modular — each automation script is a standalone, patchable function.

Desktop Vision Loop & OCR

The PC's `vision.py` module captures a screenshot every 60 seconds and sends it to the Mac Mini's `qwen3-vl:7b` model for analysis. On demand, Viper can ask "what am I looking at?" for an instant screen description. The vision model can read and react to text on screen via OCR — useful for extracting text from videos, locked documents, or any UI element. Study guard uses this loop to detect distraction apps.

Gaming Mode

When the vision loop detects a game is running, Calista automatically enters Gaming mode. She goes quiet for non-urgent matters, holds all non-critical notifications, and gives Viper a catch-up summary when the game closes. Optionally, if she recognizes the specific game from the screenshot, she can offer tips or lookup game-specific information on request.

Clipboard Memory

Calista silently monitors the clipboard during an active session and keeps the last 10 copied items in short-term session memory. "What did I copy earlier?" retrieves them. Session memory clears when the PC client disconnects. No clipboard data is persisted to disk.

Coding Assistant

Screenshot the code editor and ask Calista to help. The vision model reads the screen, the Think tier model reasons about the code, and she explains the error and suggests a fix — no copy-pasting required. This works for any language visible on screen.

Kill Switch

Ctrl+Shift+X immediately halts all running automation mid-task. Moving the mouse to the top-left corner of the screen triggers pyautogui's built-in FAILSAFE as a secondary emergency stop. Both are always active when any automation is running.

4.6 Communication & Messaging

Telegram Bot — Primary Phone Channel

Two-way communication with Calista from any phone via Telegram. All features that do not require the PC work from Telegram: chat, reminders, email management, calendar, tasks, alarms, web search, music control (for local/network playback), content summaries, and push notifications. Telegram is the fallback delivery channel for all alerts when voice is off.

Discord Bot

The Discord bot carries over from V1 with improvements. It runs on the Mac Mini permanently. Features include server chat with personality, user profiles with trust and bond tracking, slash commands (/status, /search, /calc, /setbond etc.), math engine for exact calculations, vision support for images sent in Discord, and moderation tools. Discord is a secondary social channel — Telegram is the primary personal channel.

Instagram Messaging — instagrapi

Calista sends Instagram DMs headlessly from the Mac Mini using the instagrapi library, which interfaces with Instagram's private API without requiring a browser or the app. Messages are sent from Viper's real account. All messages are drafted first and require approval before sending. An automated tag is appended to all outgoing messages so recipients know it was sent by Calista.

Draft & Approve Flow

Every outgoing message on any platform — email, Instagram, Discord, Telegram to others — goes through a mandatory draft-and-approve step. Calista composes the message, shows it to Viper, and waits. Viper can approve, edit, or cancel. Nothing is ever sent without explicit approval.

Seamless Context Handoff

A conversation started on Telegram from the phone continues naturally on the desktop without re-explaining context. The Mac Mini holds the full conversation state centrally — the PC client and Telegram bot are simply two interfaces into the same session. Switching devices mid-conversation is invisible.

4.7 Room Security (Phase 3)

NOTE All features in this section require physical room hardware and are planned for Phase 3 after the Mac Mini and room setup are complete.

Door Camera & Face Recognition

An IP camera mounted at the room entrance streams live video to the Mac Mini via RTSP. The security.py module runs face detection on each frame using the face_recognition Python library (built on dlib). Face encodings are stored in irl_people.json alongside access levels and names.

Remote Live Feed — View From Anywhere

The door camera feed is accessible from Viper's phone from any location in the world — not just on the home network. This is handled via a dedicated streaming relay running on the Mac Mini:

- The Mac Mini runs a lightweight RTSP-to-HLS or RTSP-to-WebRTC relay (MediaMTX — free, open source, zero cloud dependency)
- A secure tunnel (Tailscale or Cloudflare Tunnel — both free tiers available) exposes the stream endpoint to Viper's phone only, authenticated by private key or Tailscale identity
- On the phone, the stream is viewable via the Telegram bot (Calista sends a live snapshot on command), or directly in a browser/VLC using the tunnel URL
- Viper can say or type "show me the camera" from anywhere and Calista sends a live frame immediately via Telegram
- For continuous viewing, the tunnel URL opens the live feed directly — no third-party cloud CCTV service, no monthly fees, no data leaving the home except through the secure tunnel

NOTE The tunnel is only active when Viper initiates a connection or requests a snapshot. The camera stream itself never broadcasts publicly — it only becomes accessible via the authenticated tunnel on demand.

Entry Greeting

When Calista recognizes Viper entering the room, she greets him by name through the room speakers and delivers a brief briefing — any messages that came in, upcoming events, or anything worth knowing. The greeting adapts to time of day and her current emotion state.

Approved Persons & Stranger Alert

Registered contacts with Trusted or Allowed access level are greeted by name on entry. An unrecognized face triggers an immediate response: Calista speaks a deterrent message through the room speakers, captures a photo, and sends it to Viper's Telegram immediately with an alert. Flagged contacts are admitted but Viper is silently notified.

Access Level	Who	Calista Behavior
Trusted	Close friends, family	Greets by name, full interaction, no alert
Allowed	Acquaintances	Greets by name, limited interaction, no alert
Flagged	Persons of concern	Admits but silently sends Telegram alert to Viper
Unknown	Unregistered face	Warning message spoken aloud + photo sent to Telegram immediately

4.8 Smart Home (Phase 3)

Smart Lights & Plugs

TP-Link Kasa and Govee devices connect over WiFi and are controlled by smarthome.py on the Mac Mini via their respective Python libraries (python-kasa, govee-api-laggat). No hub required. Controlled by voice command, scene preset, or automatic trigger.

Scene & Mood Presets

- Study mode — dims ambient lights, mutes non-essential smart devices, enables study guard
- Gaming mode — sets mood lighting, holds non-urgent notifications
- Goodnight mode — turns off all devices, dims to zero, locks the room
- Morning mode — gradual light increase synced with alarm fade-in
- Focus mode — maximum brightness, fan on if registered, DND active

Context-Aware Room Automation

Calista adjusts the room automatically based on what she knows is happening. If a study session is scheduled in the calendar, she prepares Study mode before it starts. If it is 11pm on a school night, she dims lights. If the gaming mode detection triggers, she switches to gaming lighting. No manual scene switching needed.

Network Watchdog

Calista pings key services (internet gateway, Discord, Google) every few minutes from the Mac Mini. If internet connectivity drops or a key service becomes unreachable, she sends an alert via Telegram (if internet is still up) or via room speakers (if LAN is still live). Runs silently in the background at all times.

5. Memory System

Calista uses three separate JSON files for memory, each serving a distinct purpose. They are stored on the Mac Mini and persist across all sessions and restarts. The three files can be cross-linked for contacts who exist in multiple contexts.

5.1 viper_memory.json — Deep Viper Profile

The most detailed memory file. Everything Calista knows about Viper lives here. No other person has this depth of memory.

Memory Field	How Set	Examples
Preferences	Passive	Prefers dark themes, uses Chrome, dislikes mornings
Schedule patterns	Passive	Usually studies 8-10pm on weekdays, games on weekends
Tasks & goals	Active	'Submit report Friday', 'learn Japanese this year'
Important dates	Active	Exams, deadlines, trips, anniversaries
App registry	Config	.exe paths for all registered apps
Emotion history	Passive	Mood trends over weeks and months
Music preferences	Passive	Genres, artists, playlist patterns
Sleep patterns	Passive	Average sleep and wake times from alarm/goodnight data
Named contacts	Both	People Viper mentions by name, linked to irl/discord files
Named facts	Both	School subjects, projects, inside references, lore elements

5.2 irl_people.json — Real World Contacts

Stores data about people who exist in Viper's physical world. Tied to the room security system for face recognition but also used for birthday reminders, messaging, and general contact context.

Field	Description
name	Full name used for greetings and references
face_encoding	Face recognition embedding generated by face_recognition library on enrollment
access_level	trusted / allowed / flagged / unknown — controls room security behavior
relationship	friend / family / acquaintance — informs how Calista references them
discord_id	Optional link to discord_users.json entry — allows cross-context awareness
birthday	Date for automated reminder system
preferred_platform	Where to message this person — Discord, Instagram, Telegram, Gmail
last_seen	Timestamp of last detected room entry (Phase 3)
notes	Any extra context Viper adds manually

5.3 discord_users.json — Discord Contacts

Stores per-user data for every person Calista has interacted with on Discord. Carried over and expanded from the V1 system.

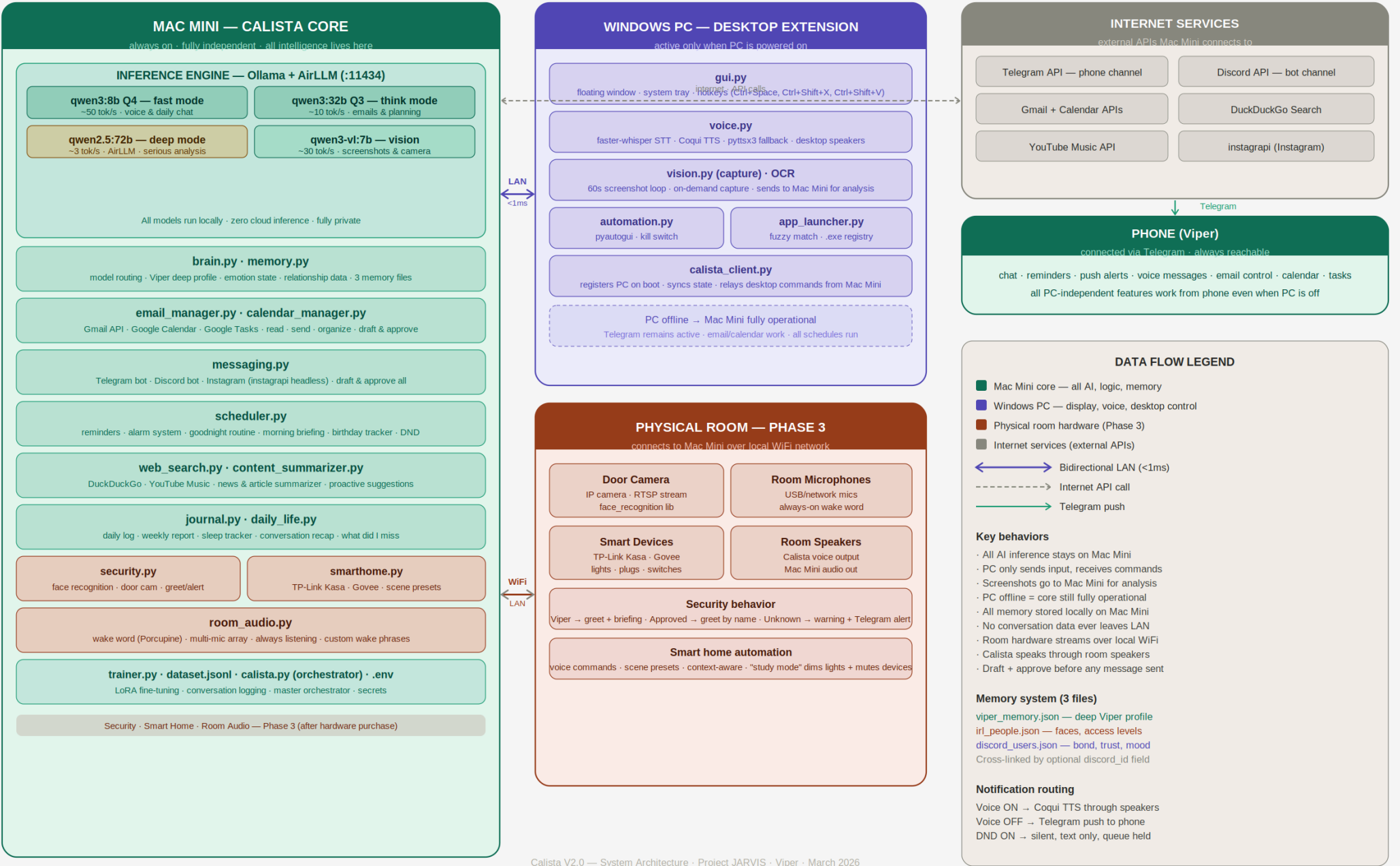
Field	Description
discord_id	Unique Discord user ID — primary key
username	Discord display name
server	Server(s) this user interacts in
trust_level	0-100 score — affects how Calista responds to requests from this user
bond_level	0-100 score — builds up from positive interactions over time
mood_history	Recent detected emotional states in messages
interaction_count	Total message exchanges logged
irl_name	Optional link to irl_people.json — enables cross-context awareness
notes	Any context Calista has built up about this person

5.4 Cross-Linking

Contacts who exist in both the real world and on Discord are linked via the discord_id field in irl_people.json and the irl_name field in discord_users.json. When Calista sees a Discord message from a linked contact, she knows they are also a real-world person Viper knows. This enables context like: "Alex just messaged you on Discord — he came over last Tuesday."

6. System Architecture

The following diagram shows the full Calista V2.0 system. The Mac Mini is the permanent, always-on core. The Windows PC is an extension that plugs in when powered on. Physical room hardware connects over local WiFi. All AI inference runs on the Mac Mini — the PC is purely for display, input, and desktop automation.



Calista V2.0 — System Architecture · Project JARVIS · Viper · March 2026

6.1 Mac Mini — Calista Core

The Mac Mini runs all AI inference via Ollama (fast and think models) and AirLLM (deep mode). It also runs all the logic modules: brain.py, memory.py, email_manager.py, calendar_manager.py, messaging.py (Telegram bot, Discord bot, instagrapi), scheduler.py, web_search.py, journal.py, security.py, smarthome.py, and room_audio.py. It is always powered on and handles every request regardless of whether the PC is on.

6.2 Windows PC — Desktop Extension

The Windows PC runs only what requires direct desktop access: gui.py (floating window and system tray), voice.py (faster-whisper STT and RVC custom voice through the PC's speakers and microphone), vision.py (screenshot capture — analysis happens on the Mac Mini), automation.py (pyautogui for desktop control), app_launcher.py, and calista_client.py which registers the PC's availability and relays commands from the Mac Mini.

6.3 File Structure

File	Purpose
calista.py	Main entry point — starts all threads and orchestrates components
brain.py	Ollama + AirLLM model routing, context management, response generation
memory.py	Read/write for all three JSON memory files, cross-linking logic
gui.py	Floating window, full window mode, system tray (Windows PC only)
voice.py	faster-whisper STT, VITS2 + RVC custom voice pipeline, voice settings panel
vision.py	Screenshot capture loop (PC), sends to Mac Mini for qwen3-vl analysis
automation.py	pyautogui desktop automation, permission flow, kill switch
messaging.py	Telegram bot, Discord bot, instagrapi Instagram — all headless on Mac Mini
email_manager.py	Gmail API — read, draft, send, organize, search, auto-label
calendar_manager.py	Google Calendar + Tasks — full CRUD, conflict detection, briefings
scheduler.py	Reminders, recurring alerts, alarm system, morning briefing, notification router
web_search.py	DuckDuckGo search, content summarizer, news queries, YouTube info

journal.py	Daily log, conversation recap, sleep tracker, weekly report
security.py	Door camera stream, face recognition, greeting/alert logic, remote live feed (Phase 3)
smarthome.py	Smart device control — TP-Link Kasa, Govee, scene presets (Phase 3)
room_audio.py	Wake word detection, multi-mic array, room STT pipeline (Phase 3)
app_launcher.py	App registry, fuzzy name matching, subprocess launcher (Windows PC)
calista_client.py	Windows PC bridge — registers with Mac Mini, relays desktop commands
trainer.py	LoRA fine-tuning script — run manually when data has accumulated
dataset.jsonl	Auto-logged conversation data for training pipeline
viper_memory.json	Deep Viper profile — preferences, schedule, contacts, emotion history
irl_people.json	Real world contacts — face encodings, access levels, birthdays, links
discord_users.json	Discord user profiles — trust, bond, mood, interaction history
voice_config.json	RVC voice model path, pitch, speed, EQ preset, named voice profiles
.env	API keys and secrets — never committed to source control

7. Build Phases & Roadmap

The 20-month roadmap is divided into three phases that build on each other. Phase 1 establishes the core AI and establishes Calista as a working assistant. Phase 2 adds desktop intelligence and full messaging. Phase 3 builds the physical JARVIS room.

Phase 1 — Core Foundation

Goal: Calista is a working, intelligent assistant accessible from phone and desktop with full memory and personality.

- Mac Mini setup — macOS, Ollama installed, all models pulled and tested
- Brain and memory system — viper_memory.json, brain.py, routing logic
- Basic chat with personality, emotion state, and dual-mode switching
- Telegram bot — two-way phone access, push notifications, all core features
- Gmail API and Google Calendar/Tasks — read, send, manage
- Scheduler — reminders, morning briefing, goodnight routine, alarm system
- Daily journal and conversation logging — dataset.jsonl begins accumulating
- Web search — DuckDuckGo, content summarizer
- Discord bot — carry over from V1 with updates
- Calista lore finalized and embedded into personality prompts

Phase 2 — Desktop Intelligence

Goal: Calista gains full awareness of the Windows PC and expands messaging to all platforms.

- Windows PC client — calista_client.py, connection to Mac Mini, hotkey system
- GUI — floating window, full window mode, system tray, voice toggle
- Custom voice pipeline — VITS2 base TTS + RVC voice model training and integration
- Desktop voice I/O — faster-whisper STT, RVC voice through PC speakers
- Desktop vision loop — 60s screenshot cycle, vision model analysis on Mac Mini
- OCR and screen reader — react to text on screen
- Study guard mode and gaming mode
- Desktop automation and app launcher
- Clipboard memory and coding assistant
- Music control — YouTube Music API and local MP3 library
- Instagram messaging via instagrapi — headless on Mac Mini
- Seamless context handoff — phone to desktop continuity
- Proactive suggestions, weather-aware notifications, event countdown
- First LoRA fine-tuning run using accumulated Phase 1 + 2 conversation data

Phase 3 — Full JARVIS Room

Goal: Calista becomes a physical room presence — aware of who enters, controlling the environment, and responding to voice from anywhere in the room.

- Room microphone array — multiple USB mics, wake word detection (Porcupine)
- Room speakers — Calista voice output throughout the room
- Door camera — IP camera, RTSP stream, face recognition pipeline
- Remote live feed — secure tunnel (Tailscale/MediaMTX) for phone access from anywhere
- Face enrollment — Viper and approved contacts registered
- Entry greeting, stranger alert, Telegram photo notification system
- Smart home integration — TP-Link Kasa, Govee, WiFi device control
- Scene presets — Study, Gaming, Goodnight, Morning, Focus modes
- Context-aware room automation — calendar-driven scene switching
- Network watchdog — service monitoring, connectivity alerts
- Sleep schedule tracker fully operational from accumulated Phase 1-3 data
- Second LoRA fine-tuning run — personality now shaped by a full year of data

8. Hardware Requirements

This section lists all hardware required to build the full Calista V2.0 system across all three phases. Phase 1 and 2 hardware is the minimum requirement to get started. Phase 3 hardware is purchased when the room build begins.

8.1 Phase 1 & 2 — Core Hardware

Component	Specification	Purpose
Mac Mini M4	16GB unified memory, 256GB SSD	Primary AI server — all inference, logic, memory, bots. Always on.
Windows PC	Existing hardware	Desktop client — GUI, voice, vision capture, automation, app launching
Network router	Any gigabit home router	Stable LAN between Mac Mini and Windows PC. Wired preferred for reliability.
Ethernet cable	Cat5e or Cat6, any length	Wired connection for Mac Mini — more stable than WiFi for always-on server
USB microphone	Any quality USB mic	Desktop voice input for the Windows PC client in Phase 2

8.2 Phase 3 — Room Hardware

Component	Recommended Options	Purpose
IP Door Camera	Any 1080p RTSP-capable IP cam (Reolink, TP-Link Tapo)	Door monitoring for face recognition, security system, and remote live feed
Room microphones	2-4x USB mics (Blue Snowball, Samson Go, etc.)	Always-on room audio array for wake word and voice commands
USB hub	Powered 4+ port USB hub	Connect multiple room mics to Mac Mini simultaneously
Room speakers	Wired or Bluetooth speakers near Mac Mini	Calista voice output audible throughout the room
Smart plugs	TP-Link Kasa EP10 or EP25 (2-4 units)	Control desk lamp, fan, PC monitor, and other devices by voice
Smart LED strip	Govee LED strip (2m or 5m)	Ambient room lighting controlled by scene presets and mood
Smart bulb(s)	TP-Link Kasa KL110 or Govee bulbs	Main room lighting with dimming and color temperature control
Mounting hardware	Camera mount, mic stands or ceiling clips	Clean installation of camera and microphones around the room

8.3 Mac Mini M4 — Technical Notes

- 16GB unified memory is shared between CPU and GPU (Metal cores). This is not a limitation — unified memory means the GPU can directly use all 16GB for model weights with zero copy overhead.
- The M4 Neural Engine accelerates both Ollama inference and AirLLM layer loading significantly compared to a standard CPU machine.
- 256GB SSD provides 7.4 GB/s read speed — fast enough to make AirLLM's layer-by-layer loading tolerable at ~3 tok/s for the 72B deep mode model.
- The Mac Mini should be connected via Ethernet to the router for maximum reliability as an always-on server.
- macOS Sequoia is recommended. Ollama has a native macOS installer from ollama.com.
- Power consumption is approximately 10-20W at idle — extremely low for a server running 24/7.

8.4 Ollama Model Storage

Model	Disk Size (approx)	Notes
qwen3:8b Q4_K_M	~5 GB	Fast mode — pulled via Ollama, always resident in memory during operation
qwen3:32b Q3_K_M	~13.5 GB	Think mode — fits in 16GB unified memory, loaded on demand
qwen2.5:72b 4-bit	~40 GB	Deep mode — stored on SSD, loaded layer-by-layer via AirLLM, slow but capable
qwen3-vl:7b Q4	~5 GB	Vision model — pulled via Ollama, used for screenshots and door camera
faster-whisper	~150 MB	STT model — downloaded once, runs on CPU, fully offline after first run
RVC voice model	~200 MB	Custom trained voice model — stored on Mac Mini, loaded into voice pipeline
Total	~64 GB	Leaves ~190 GB free on the 256GB SSD for OS, apps, datasets, and training

8.5 Python Packages (Full Install List)

Package	Purpose
ollama	Ollama Python client for local model inference
airlm	AirLLM for layer-by-layer 72B deep mode inference
faster-whisper	Offline speech-to-text transcription (STT)
vits / so-vits-svc	VITS2 base TTS for custom voice pipeline
rvc-project	RVC voice conversion — custom Calista voice model inference and training
pyttsx3	Emergency offline TTS fallback — system voices only
python-telegram-bot	Telegram bot for phone access and notifications
discord.py	Discord bot for server chat and slash commands
instagrapi	Instagram headless messaging via private API
google-api-python-client	Gmail and Google Calendar/Tasks API
google-auth-oauthlib	OAuth authentication for Google APIs
duckduckgo-search	Web search — no API key required, completely free
ytmusicapi	YouTube Music control — search and queue tracks
mediamtx	RTSP relay for door camera remote live feed (Phase 3)
tailscale	Secure tunnel for remote camera access from any network (Phase 3)
face_recognition	Face detection and recognition from door camera (Phase 3)
python-kasa	TP-Link Kasa smart plug and bulb control (Phase 3)
govee-api-laggat	Govee LED strip and smart device control (Phase 3)
pvporcaupine	Picovoice Porcupine wake word engine — offline always-on detection (Phase 3)
pyautogui	Desktop mouse and keyboard automation (Windows PC)
pystray	System tray icon and context menu (Windows PC)
pillow	Screenshot capture, image processing
requests / httpx	HTTP client for API calls and webhooks
python-dotenv	Load secrets from .env file securely
peft / unsloth	LoRA fine-tuning pipeline for personality training
sympy	Math engine for exact numerical answers
psutil	System monitoring for optional PC health checks